# Reinforcement Learning for Long-Context Reasoning

## Hierarchical Agent Architecture with Decomposed Rewards

Arjun Vaidya  Peter Driscoll

May 2026

# The Problem

## Mathematical Reasoning Challenge

Large language models struggle with multi-step reasoning [1].
Chain-of-thought helps, but better training methods are needed.

## Our Solution

Use RL to incentivize the model towards the right distribution of reasoning and solving behaviors, without a separate value network.

**Methods:**
Hierarchical decomposition, GRPO [2], decomposed rewards

## The Bigger Challenge

Prior work (CoT [1], GRPO [2], hierarchical agents [3]) show promise.

**Challenge:**
Decoupling rewards between reasoning and solving phases while maintaining signal flow.

# Dataset: GSM8K

## Overview

- 8.5K grade-school math word problems.
- Linguistically diverse prompts requiring multi-step reasoning.
- Answers include chain-of-thought style reasoning plus a final numeric target.

## Data Structure

- **Question:** math word problem.
- **Answer / reasoning:** step-by-step solution logic.
- **Final answer:** numeric label, often marked with `####`.

## How We Use It

- Fine-tune on the `train` split.
- Extract exact numeric answers for automated grading and reward computation.

# Baseline: Reward-Weighted Supervised Fine-Tuning

## Flat Baseline

- Standard causal language model.
- No routing or expert-selection mechanism.
- Custom `RewardWeightedTrainer` for supervised fine-tuning.

## Efficiency Choice

- LoRA adapters target attention projections.
- Main targets: `q_proj` and `v_proj`.

## Training Objective

1. Format examples as: `Question: [Q] \n Reasoning: [A]`.
2. Assign binary reward from numeric-answer format.
3. Weight sequence cross-entropy by reward.

$$\mathcal{L} = R(x, y)\, \mathcal{L}_{\mathrm{CE}}(x, y)$$

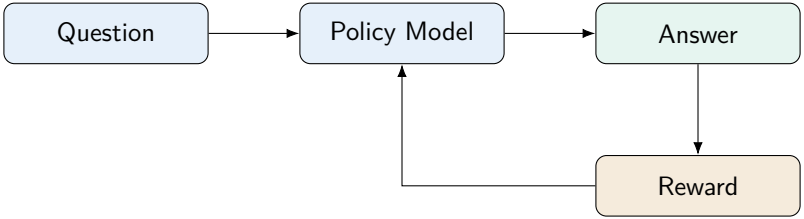
# Why Direct RL Over SFT?

## Computational Efficiency

- Skip an intermediate SFT stage.
- Optimize directly against task reward.
- Reduce mismatch between training objective and evaluation objective.

## Alignment Benefits

- Avoid overfitting to human-curated SFT traces.
- Learn from the reward signal that actually defines success.
- Useful when correct outputs can be automatically checked.



# GRPO: Eliminating the Critic Network

## Traditional PPO

- Requires a separate critic network.
- Critic estimates per-step value functions.
- Adds memory, compute, and tuning overhead.

## GRPO Idea

- Sample a group of responses for the same prompt.
- Compare within-group and normalize rewards.

$$A_i = \frac{R(r_i) - \mathrm{mean}(R)}{\mathrm{std}(R)}$$

**No value model**

**Lower overhead**

**Sequence-level credit**

# Router Reward: Outcome-Only vs. Process-Based

## Outcome-Only Reward

- $R = 1$ if final answer matches ground truth.
- $R = 0$ if final answer is incorrect.
- Easy to implement and cheap to compute.
- Weakness: gives no feedback on intermediate planning quality.

## Process Reward Models

- Score each planning step independently.
- Evaluate clarity, concreteness, and logical order.
- Can detect poor plans before final execution fails.
- Weakness: requires step-level quality metrics.

**Core Tradeoff**

Outcome rewards are simple but sparse. Process rewards are richer but harder to define without adding noise or reward hacking.

# Conditional and Hybrid Router Rewards

## Conditional Reward Modeling

- Scores a step based on preceding steps and the final outcome.
- Asks whether the plan would work if executed perfectly.
- Checks for necessary quantities, correct decomposition, and logical order.
- Captures inter-step dependencies but makes credit assignment more complex.

## Practical Hybrid Reward

$$R_{\text{hybrid}} = 0.5\, R_{\text{plan}} + 0.5\, R_{\text{outcome}}$$

- Balances structure and correctness.
- Gives smoother learning signal than outcome-only reward.
- Lowest-cost starting point for router reward development.

# LLM-as-Judge: Expert Plan Evaluation

## How It Works

1. Router generates a structured plan.
2. Send question and plan to Claude with an evaluation prompt.
3. Score clarity, completeness, and feasibility from 0 to 10.
4. Convert to $[0, 1]$ and blend with outcome reward.

## Strengths and Limits

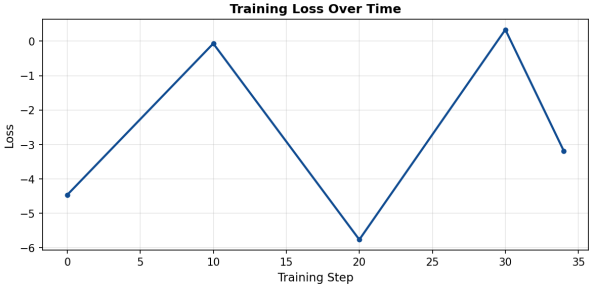
- **Strength:** catches subtle plan errors missed by heuristics.
- **Strength:** no reward model training required.
- **Limit:** slower because it requires API calls.
- **Limit:** cost and non-determinism make it risky for high-volume training.

## Practical Strategy

Train with fast heuristic rewards and validate with an LLM judge to detect reward inflation, reward deflation, and brittle heuristic behavior.

# Training Results & Key Challenges

1. **Memory Constraints:** Long-sequence decoding (512+ tokens) causes OOM. Hierarchical agent multiplies the problem.
2. **Solution:** Per-rollout backward passes (B=1, G=2) and CodeBatcher for parallel CPU code execution.
3. **RL Training:** Reward design and group size heavily impact convergence. Evaluation cost dominates.



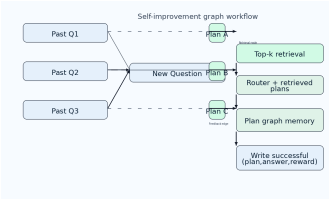
# Future Work: Graph for Self-Improvement

## From original proposal

*A persistent key-value store of (`problem_embedding` → `successful_plan`) pairs. The architecture has one deliberate extension hook where the Router prompt can prepend retrieved plans from similar solved problems.*

## Proposed self-improvement graph

- Nodes are problems, plans, and rewards in one retrieval graph.
- Edges combine embedding similarity with success metadata.
- Router reads top-$k$ successful, similar plans before generation.
- Write-back triggers only on clean successes (correct + valid plan + no tool errors).



# References


1 Wei, J., Wang, X., et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS 2023. https://arxiv.org/abs/2201.11903

2 DeepSeek Team. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. https://arxiv.org/pdf/2501.12948

3 Weng, L. *LLM Powered Autonomous Agents*. Blog Post, 2023. https://lilianweng.github.io/posts/2023-06-23-agent/

4 Zhang et al. *ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search*. NeurIPS 2024. https://arxiv.org/abs/2406.03816

5 *Linking Process to Outcome: Conditional Reward Modeling for LLM Reasoning*. https://arxiv.org/abs/2509.26578